

Building the AArch32 and AArch64 ports

Everything needed to go from empty directory to twenty-four test images per architecture, running under QEMU or on a Raspberry Pi.

1. Get the sources

Each architecture project keeps **no copy** of the kernel or the drivers. Clone what you need side by side:

```
mkdir workspace && cd workspace
git clone <remote>/micro-os-plus-iii-smp.git
git clone <remote>/micro-os-plus-iii-devices.git
git clone <remote>/micro-os-plus-iii-aarch64.git      # and/or -aarch32
```

```
workspace/
├─ micro-os-plus-iii-smp/
├─ micro-os-plus-iii-devices/
├─ micro-os-plus-iii-aarch64/
└─ micro-os-plus-iii-aarch32/
```

CMake finds them as siblings. Override either with

```
cmake -DUOS_SMP_DIR=/elsewhere/micro-os-plus-iii-smp \
      -DUOS_DEVICES_DIR=/elsewhere/micro-os-plus-iii-devices ...
```

A build that cannot find one stops and names the repository to clone.

One kernel working copy serves every architecture project on the machine, so an edit to it is visible to all of them at once — no submodule pointer to bump, no commit-push-pull round trip.

2. Tools

Tool	Used for	Notes
arm-none-eabi-gcc 15	aarch32	xPack
aarch64-none-elf-gcc 15	aarch64, and the AArch32 QEMU boot shim	xPack
CMake ≥ 3.20	both	
qemu-system-aarch64	both suites	xPack QEMU 9.x
python3 + markdown, weasyprint	docs/md2pdf.py	docs only

Toolchains are found under ~/.local/xPacks/@xpack-dev-tools/, newest first, falling back to PATH. Pin one exactly with

```
cmake -DUOS_TOOLCHAIN_BIN=~/.local/xPacks/@xpack-dev-tools/aarch64-none-elf-gcc/15.2.1-1.1.1/.cont
```

3. Configure and build

AArch64 — Raspberry Pi Zero 2 W / Pi 3 B, 4× Cortex-A53, ARMv8-A

```
cd micro-os-plus-iii-aarch64
cmake -S . -B build \
      -DCMAKE_TOOLCHAIN_FILE=../micro-os-plus-iii-smp/cmake/toolchains/aarch64-none-elf.cmake
cmake --build build -j8
```

AArch32 — same silicon in 32-bit mode, ARMv7-A

```
cd micro-os-plus-iii-aarch32
cmake -S . -B build \
      -DCMAKE_TOOLCHAIN_FILE=../micro-os-plus-iii-smp/cmake/toolchains/arm-none-eabi.cmake
cmake --build build -j8
```

Board

AArch64 builds for the Pi only. AArch32 carries three boards:

```
cmake -S . -B build      -DBOARD=zero2w ... # default
cmake -S . -B build-rpi3b -DBOARD=rpi3b ...
cmake -S . -B build-lyra -DBOARD=luckfox-lyra ...
```

	zero2w	rpi3b	luckfox-lyra
silicon	BCM2837	BCM2837	RK3506
SMP cluster	4× Cortex-A53 (32-bit)	4× Cortex-A53 (32-bit)	3× Cortex-A7
-mcpu	cortex-a53	cortex-a53	cortex-a7
-mfpv	neon-fp-armv8	neon-fp-armv8	neon-vfpv4
board dir	boards/rpi-zero-2w	boards/rpi-zero-2w	boards/luckfox-lyra
define	LED_PIN=29	BOARD_RPI3B	SOC_RK3506
OS_NCPU	4	4	3
applications	12	12	7
QEMU machine	raspi3b + shim	raspi3b + shim	virt, no shim

rpi3b selects linker-rpi3b.ld and defines BOARD_RPI3B.

luckfox-lyra is the RK3506's three Cortex-A7. Its Cortex-M0 is a different ISA outside the coherency domain and is not a CPU the scheduler can use, so the board is multi-core while the SMP cluster is three. It builds the seven applications that need only CPUs, a timer and a console; the other five link micro-os-plus::devices, whose SD and USB backends are BCM2837-specific.

Adding it moved three files up into the shared port, and turned up seven places where one board had been mistaken for the architecture. See aarch32-second-board.md.

What you get

24 targets — twelve applications × {qemu, hwd} — each producing an ELF, a .bin (the kernel8.img / kernel7.img equivalent) and a .map, all in build/test/.

sd_test-qemu	sd_test-hwd
smp_test0..4-qemu	smp_test0..4-hwd
smp-mat-test-qemu	smp-mat-test-hwd
smp-mat-sdcard-test-...	smp-num-test-...
smp-pipeline-test-...	smp-pro-cons-test-...
usb_test-qemu	usb_test-hwd

qemu is the emulator build. hwd adds HW_BUILD (SD tests use the existing FAT32 boot partition rather than formatting a blank card) and DEBUG_BOOT (early-boot asm markers for OpenOCD bring-up).

The shared suite itself — what each of the twelve applications proves, how apps.cmake configures them, and how to add one — is test-smpl.md.

4. Running the QEMU suites

One runner serves every architecture; the caller supplies the machine.

AArch64 — straight to -kernel

```
../micro-os-plus-iii-smp/test_smpl/run-qemu.sh build/test \  
"$ (ls ~/.local/xPacks/@xpack-dev-tools/qemu-arm/*/.content/bin/qemu-system-aarch64 | tail -1) \  
-M raspi3b -smp 4
```

AArch32 — through the boot shim

QEMU's raspi3b starts its Cortex-A53 cores in AArch64, so a 20-line stub drops to AArch32 and jumps to the image. CMake builds it as qemu-shim.

```
UOS_QEMU_SHIM=build/test/shim8.img UOS_QEMU_LOAD_ADDR=0x10000 \  
../micro-os-plus-iii-smp/test_smpl/run-qemu.sh build/test \  
"$ (ls ~/.local/xPacks/@xpack-dev-tools/qemu-arm/*/.content/bin/qemu-system-aarch64 | tail -1) \  
-M raspi3b -smp 4
```

Not raspi2b. The port is built -mcpu=cortex-a53, so its load-acquire instructions are undefined on the raspi2b Cortex-A7 and the image faults at the first one. Hardware needs no shim at all: config.txt sets arm_64bit=0 and the cores come up in AArch32 directly.

The runner creates the SD images the four card tests need — a seeded flatfs volume for sd_test, a blank 4 GiB image for the others — under build/test/.qemu-logs/.

Run one suite at a time. Two four-core suites at once, or one alongside a build, starves a QEMU vCPU: a worker freezes mid-loop with no fault, which looks exactly like a deadlock. Re-run on an idle host before investigating.

Result

qemu suite: 11 passed, 1 skipped, 0 failed

on both architectures. usb_test skips by design — QEMU emulates no USB device mode — exactly as the predecessor suite recorded it.

5. Running on hardware

Measured on a Raspberry Pi Zero 2 W over a SEGGER J-Link.

Port	Hardware
AArch64	all twelve tests pass
AArch32	all twelve tests pass

Each figure is from runs on that port. They are not inferred from one another: the two ports share every test source, but they are separate binaries, and a stale build is indistinguishable from a bug until you rebuild and re-run. **Rebuild the port you are about to test.**

Standalone boot

Flash a *-hwd.bin as kernel8.img (AArch64) or kernel7.img (AArch32) on the boot partition, with the port's config.txt from boards/rpi-zero-2w/.

Debug-in-RAM, through OpenOCD

```
cd micro-os-plus-iii-aarch64          # or -aarch32  
test/hw.sh list                       # the tests this build has, and their budgets  
test/hw.sh smp_test2                 # run one
```

test/hw.sh holds only what is specific to the port and the board — the binutils, the entry fallback, the OpenOCD target names, which cores are debug targets at load time, the width of __smp_spin, how a core is resumed, and whether anything has to happen between the halt and the load. The session is driven by test_smp1/run-hw.sh in the kernel repository, which every port shares. Together they replace the predecessor's hw.sh + hw-olimex.sh in each test directory of each port: 48 files of about 200 near-identical lines.

It is **pure OpenOCD** — no GDB, no reset, and it never opens the serial device, so it cannot fight the terminal you keep on the console. Keep your own `tio -b 115200 /dev/ttyACM0` running. Everything OpenOCD and the board's semihosting write appears on your terminal as it happens; the copy teed to `build/test/.hw-logs/<app>.log` exists only so the script can match the verdict.

One test per power cycle. Every run is `load_image` into RAM over whatever the previous test left there, and neither board has a reset a script can drive — the Pi has no SRST and its Cortex-A53 debug target has no reset method, and the Lyra's secondaries are released once, by clearing their CRU reset bits, which a second run cannot undo. So `run-hw.sh` takes exactly one test and refuses a suite. Power-cycle between tests.

Env	Meaning
BOARD	zero2w (default), rpi3b or luckfox-lyra jlink (default) or olimex — the Pi boards only the CMake build directory (default build, or build-lyra)
PROBE	
BUILD	
UOS_HW_ADAPTER_KHZ	override the JTAG clock. board/rpi3.cfg asks for 4000 kHz, more than jumper wires always carry. A DAP that gives up mid-run prints Invalid ACK (0) in DAP response and then fails to re-examine every core; the runner reports that as DEBUG LINK LOST , not as a firmware fault.

The third argument overrides the run budget in seconds. A budget is **not** the test's own duration: it is dominated by semihosting traps, and those scale with how much a test prints. `smp_test4` reaches its verdict at `t=9597` ms of target time yet needs well over 120 s of wall clock, because its reporter emits about nine lines a second and each is a debug halt and resume over JTAG.

On a Luckfox Lyra B

```
cd micro-os-plus-iii-aarch32
cmake -S . -B build-lyra -DBOARD=luckfox-lyra \
      -DCMAKE_TOOLCHAIN_FILE=../micro-os-plus-iii-smp/cmake/toolchains/arm-none-eabi.cmake
cmake --build build-lyra -j8
```

```
BOARD=luckfox-lyra test/hw.sh list           # the seven tests, and their budgets
BOARD=luckfox-lyra test/hw.sh smp_test0     # run one
```

Same runner, same rules, three differences — all of them in `boards/luckfox-lyra/openocd.cfg` and the `luckfox-lyra` branch of `test/hw.sh`, none of them in the shared runner:

The probe is a WCH-Link over SWD, not a J-Link over JTAG — `cmsis-dap`, USB `1a86:8011`, reset_config none separate, 1000 kHz. There is no `PROBE` choice on this board.

Only core 0 is a debug target when the image is loaded. Cores 1 and 2 sit in the BootROM until the kernel clears their CRU reset bits, so the board's OpenOCD config declares them `-defer-examine` and the runner drives core 0 alone (`UOS_HW_CORES=0`). There is no `__smp_spin` to zero either: the secondaries are released through the SRAM mailbox, not a spin table.

The MMU and caches have to be turned off before the load. The Rockchip miniloader hands core 0 over with both on, and load_image writing through a dirty cache leaves DRAM holding something other than the image. hw.sh supplies the SCTLR.{M,C,I} clear + I-cache/BP/TLB invalidate as UOS_HW_PRELOAD, which is the same sequence the board's own write_board.sh has always used.

The console is the Lyra debug header at **115200**, the rate the miniloader leaves UART1 at. The port does not reprogram it unless the build defines UART_BAUD (-DUART_BAUD=1500000 gives the exact divisor-1 rate off the 24 MHz sclk_uart1). Keep your own terminal on it; the runner never opens it, and the verdict is read from the semihosted console in OpenOCD's log.

Power-cycle first, every time. The prompt in write_board.sh was not a formality: releasing cores 1 and 2 clears their reset bits, and nothing short of a power cycle puts them back. A second run without one finds them already out of the BootROM and running whatever the last test left behind.

Not yet measured: no test in this repository has been run on a Lyra. The seven builds exist and the session is wired; the results table above covers the Pi only.

The boot card

The four SD tests and usb_test mount the card's **existing** FAT32 partition through FatFs and keep every file under tests/. They never format it and never touch the root. boards/rpi-zero-2w/verify-bootcard.py proves that byte-for-byte under QEMU.

FatFs is built without long-name support, so a file arrives on the card as an 8.3 name in upper case — xfer.bin is stored as XFER.BIN, and the device says so.

usb_test needs a host

It is the only test QEMU cannot run at all, and the only one that needs something done on the host while it runs.

Power the board from **PWR IN** and run the data cable from the PC to the **USB** socket — the OTG port. On the wrong socket the board boots and prints normally but never enumerates, with no error message.

```
test/hw.sh usb_test 900           # one terminal
# ~20 s later, in another:
cd ../micro-os-plus-iii-smp/test_smpl/common/usb_test
sudo ./host_xfer.py
```

With no arguments host_xfer.py runs the size matrix — 0, 1, 511, 512, 513, 1024, 65536, 1048576 bytes, each written and read back with CRC checks — then a LIST, then the terminating PING that makes the device print its tally and RESULT. Without that ping the device serves commands for ever and never reaches a verdict. send_file.py is the single-file demo and terminates only with --terminate.

The verdict is crc_errors == 0 && commands != 0 && active >= 3: at least three of the four cores must have run the unpinned load generators, which is what makes it an SMP test and not only a USB one.

Defects this suite found

Hardware exercised paths QEMU cannot, and five of them were real:

Defect	Why only on hardware
Every blinking test drove GPIO16	led.hpp falls back to header pin 36; the Zero 2 W's onboard ACT LED is GPIO29. Only usb_test had ever set LED_PIN. It is a board fact now, set once for every test.
usb_test replied to a PUT after its console output	The reply is sent immediately after the store now. The hexdump and listing are queue posts drained one semihosting trap at a time, so a completed store looked like a hang.
Console posts blocked the USB service thread	They use try_send now and are dropped when the consumer falls behind, with console_dropped in the tally. Diagnostics must never throttle the protocol.
LIST compared names case-sensitively	fatfs under QEMU keeps the name as sent; FatFs on the card returns XFER.BIN.
kOutChunkMax overflowed PKTCNT	D{I,0}EPTSIZ is bounded by <i>two</i> fields. This core reports GHWCFG3 = 0x0ff000e8: XFRSIZ 19 bits (524287 bytes) but PKTCNT 10 bits — 1023 packets , so 65472 bytes at full speed. The old 0x7F000 was sized against XFRSIZ alone and against 512-byte packets, and asked for 8128 packets in a 10-bit field. A 1 MiB transfer stored 61440 bytes, truncated. 65536 was the last size to pass because it overflows the field by exactly one packet.

DEBUG_BOOT is off by default in the hwd builds, as the sources assume — each early-boot marker is a semihosting trap. -DUOS_DEBUG_BOOT=ON restores them for board bring-up.

6. Declaring your own application

```
uos_add_app (my_app
  SOURCES      main.cpp
  PORT         micro-os-plus::aarch64
  LIBRARIES     micro-os-plus::devices
  LINKER_SCRIPT ${CMAKE_CURRENT_SOURCE_DIR}/linker.ld
  NCPU         4
  DEFINES      TRACE __ARM_EABI__ SEMIHOST
)
```

uos_add_app links the kernel, sets OS_NCPU (and OS_USE_SMP_SCHEDULER when it exceeds 1), applies the bare-metal flag set, adds the linker script, and emits the .bin and a size listing. It replaces the 214 Makefiles the predecessor repository carried, which differed by about 48 lines of boilerplate each.

To build one of the shared tests instead:

```
uos_add_test_app (smp_test1 NCPU 4 LINKER_SCRIPT ... DEFINES ...)
```

which resolves the sources from test_smpl/common/smp_test1/. No build ever spells out a path into the test tree.

7. CMake targets exported

micro-os-plus-iii-smp

Target	Contents
micro-os-plus::iii	the kernel core: 37 sources every port compiles
micro-os-plus::iii-posix-io	13 sources, the file-descriptor layer
micro-os-plus::iii-drivers	2
micro-os-plus::iii-startup	the generic reset path — a port with its own startup.S skips it
micro-os-plus::iii-newlib-reent	1
micro-os-plus::iii-semihosting	1
micro-os-plus::iii-trace-itm / -trace-semihosting / -trace-segger-rtt	mutually exclusive; link at most one
micro-os-plus::port-smp-decls	the shared os-decls.h
micro-os-plus::test-common	the shared test support

The ARM ports link the core, port-smp-decls and test-common, and **none** of the optional groups. Linking iii-posix-io into a newlib bare-metal build fails to compile: it declares read/write returning ssize_t where newlib declares int.

micro-os-plus-iii-devices

Target	Contents
micro-os-plus::devices	SD, flatfs, DWC2, FatFs
micro-os-plus::soc-bcm2837	BCM2837 registers, IRQ, mailbox

The architecture projects

micro-os-plus::aarch32, micro-os-plus::aarch64 — each links micro-os-plus::iii, micro-os-plus::port-smp-decls and micro-os-plus::soc-bcm2837, and carries its -mcpu as both a compile and a **link** option. Without the machine flags at link time the driver picks the wrong multilib and every AArch32 link fails with “uses VFP register arguments”.

8. Defines

See smp-construction.md §5 for the complete table, per folder: what the port header fixes, what uos_add_app sets, what each architecture project’s test/CMakeLists.txt adds, and which of the kernel’s many optional OS_* switches matter (none of them, for these builds).

9. Rebuilding this document

```
python3 docs/md2pdf.py docs/building-aarch32-aarch64.md docs/building-aarch32-aarch64.pdf \
  --title "Building the AArch32 and AArch64 ports" \
  --subtitle "μOS++ III SMP" \
  --footer "μOS++ III SMP — build guide"
```